CMPSC 132 Final Project Report — Tic-Tac-Toe
Sergio Yacolca

1. Project Description

This project involves the development of a terminal-based Tic-Tac-Toe game using Python. The program allows two players to take turns making their moves until one of them wins or the game ends in a draw, when all the squares are occupied. In addition to meeting the basic requirements of the game on a 3×3 board, the program includes extra features such as 4×4 and 5×5 boards, offering two game modes: Classic Mode and No-Draw Mode, symbol selection (including emojis), and a system for saving and loading games.
Additionally, color was implemented in the terminal to improve visibility and make the game more dynamic and interactive. Finally, the project includes a GitHub repository containing all the source code, along with a README.md file that provides clear instructions for use, explanations of features, and project details, allowing any user to easily understand and run the program.

2. Design and Implementation

The program was designed using a modular approach, employing multiple functions that clearly separate the game logic. This keeps the code organized, reusable, and easy to understand.
The main function is **play_game**, which controls the entire game flow: turns, board updates, verification of win or draw conditions, and execution of special commands such as ‘save’, ‘undo’, and ‘exit’. User input is handled by **get_move**, which validates the entered data, preventing errors and invalid moves.
The winning logic is implemented in the **check_winner** function, using a sliding window technique that allows for the verification of winning combinations on boards of any size. This makes the program scalable and independent of fixed conditions, unlike in traditional Tic-Tac-Toe.

Additionally, a clear separation of responsibilities was implemented:

- Display functions (**print_board, print_reference_board**)
- Logic functions (**check_winner, is_draw**)
- Control functions (**play_game, switch_player**)
- Persistence functions (**save_game, load_game**)

A simple state machine was also added to the initial menu using the step variable, allowing navigation between different stages (size, names, symbols, starting turn) without restarting the program.

3. Course Concepts and Data Structures Used

This project directly applies multiple concepts taught in the CMPSC 132 course, particularly in the design of structured programs and the use of data structures.

- Lists and nested lists: used to represent the board as an NxN matrix, allowing for efficient access to and modification of each cell.
- Dictionaries: used to store key information such as player names, symbols, scores, and save data. This allows the data to be organized in a clear and accessible way.
- Tuples and lists: used in the move history, where each move is stored as (player, row, col). This facilitates the implementation of features such as undo.
- Flow control (**if, while, for**): essential throughout the program, especially for input validation, game loops, and condition checking.

- Modular functions: each part of the program is separated into specific functions, following design best practices covered in class.
- Error and exception handling (**try/except**): applied when reading inputs to prevent the program from crashing due to invalid inputs.
- Files and JSON: used to implement data persistence (saving and loading games), applying file handling concepts covered in the course.
- Code abstraction and reuse: functions like **rebuild_from_history** allow the game state to be reconstructed without duplicating logic.

Overall, the project demonstrates the ability to design a complete system using fundamental concepts from the course, combining them to solve a real-world problem.

4. Challenges Faced

One of the main challenges was implementing victory detection for different board sizes. In a traditional 3×3 Tic-Tac-Toe game, winning combinations can be hard-coded. However, when including 4×4 and 5×5 boards, it was necessary to create more flexible logic that could check rows, columns, and diagonals depending on the board size and the number of symbols needed to win.
Another challenge was ensuring the board displayed correctly when players chose emojis as symbols. Some emojis take up more visual space on the terminal than a normal letter, which could cause the board to become misaligned. To resolve this, the program detects whether symbols take up more space and dynamically adjusts the width of the cells.
Implementing the undo system was also complex. Simply undoing the last move wasn't enough, especially in No-Draw Mode, where old pieces can disappear automatically. Therefore, the program saves the move history and can correctly reconstruct the board after undoing moves.
In addition, the save and load system presented a challenge because the data had to be stored in a JSON file without losing the game state. It was necessary to save data such as the board, players, symbols, score, game mode, and move history, and then correctly restore it when loading a game.
Finally, implementing the "back" option during game setup also required careful attention, as the user can return to the previous step without restarting the entire program. This made the menu flow more flexible and user-friendly.

5. Future Improvements

As for future improvements, implementing a single-player mode against the computer with artificial intelligence could also include different difficulty levels to make the game more dynamic.
It would also be possible to develop a graphical user interface (GUI), which would significantly improve the visual experience and make the game more accessible. This is one of the improvements I would be most interested in implementing, as I believe that providing a better visual experience is the most important aspect.
Additionally, more advanced statistics could be added, such as match history, win rates, or player performance analysis.
Another improvement would be to optimize the save system, allowing for multiple user profiles or more structured storage. I believe this would result in an experience closer to that of a high-quality game.
Finally, code efficiency could be improved in certain areas, particularly in victory detection, by applying complexity analysis and algorithm optimization.